



SCHOOL OF COMPUTING AND CREATIVE MEDIA
BACHELOR OF COMPUTER SCIENCE (HONS)
CSS3143 ALGORITHM AND DESIGN COMPLEXITY
FEBRUARY 2026

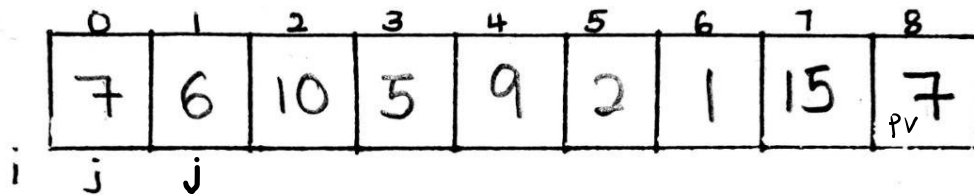
ASSIGNMENT (Individual Work)

STUDENT NAME	STUDENT ID
ELLENA CHRISTIE ANAK NULANG	BCS24090025

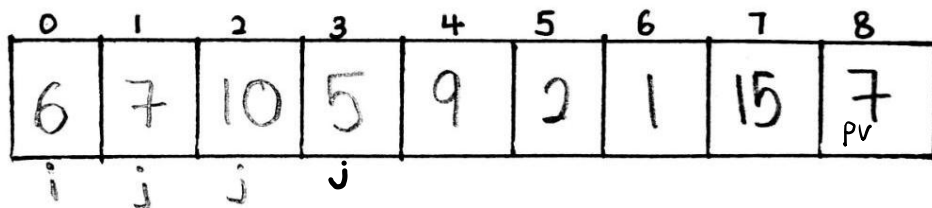
SUBMISSION DATE	2 APRIL 2026
LECTURER	DR. TANALACHIMI GANAPATHY

1. Please explain how Quick Sort, Heap Sort, and Shell Sort works. Please illustrate and explain with the help of diagram.

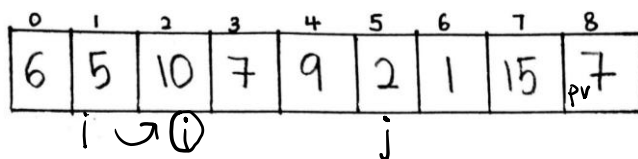
Quick Sort



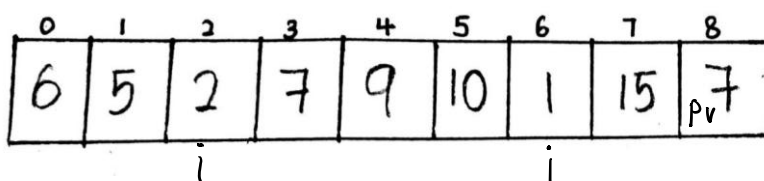
- Choose a **pivot**, which here we choose 7 in [8] as pivot
- **i** and **j** are pointers, and we start **j** at [0] and **i** before [0], **j** will keep moving and no changes happen when number are more or equal than pivot while **i** will do an increment if number are less than pivot
- start from comparing 7 with pivot, so $7 = 7$ means that we do not swap anything
- **j** move to [1] while **i** stays, $6 < 7$ so **i** are increased to [0] and swap number on [0] with 6



- **j** move to [2] while **i** stays at [0]
- $10 > 7$ so do not swap anything and **j** moves to [3]
- $5 < 7$ so **i** are increased to [1] and swap number on [1] with [3]



- now **i** is at [1] while **j** keep moving to [2],[3],[4] and do not swap as all those numbers are larger and equal to pivot
- **j** at [5] at $2 < 7$ so **i** make increment to [2] and swap number 10 with 2



- now **i** stay at [2] and **j** moving from [5] to [6]
- $1 < 7$ so **i** increased to [3] and number 7 swap with 1

0	1	2	3	4	5	6	7	8
6	5	2	1	9	10	7	15	7
			i				j	pv

- now **i** stay at [3] and **j** moves to [7]
- $15 > 7$ so do not swap anything
- Lastly we need to **swap pivot with elements (i+1)** which is 9 at [4] so the elements from **left side is less than pivot while elements on right side is larger than pivot**
- Now the pivot 7 is now permanently locked in its final, sorted position.

0	1	2	3	4	5	6	7	8
6	5	2	1	7	10	7	15	9
				i				

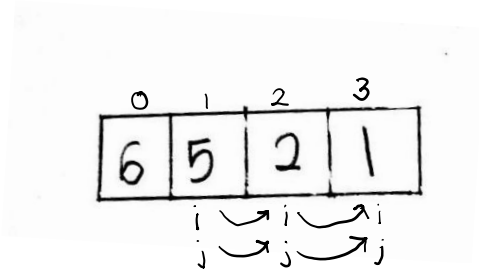
- Now separate elements set that is less than pivot=7 and elements set that is more than pivot=7

6	5	2	1
---	---	---	---

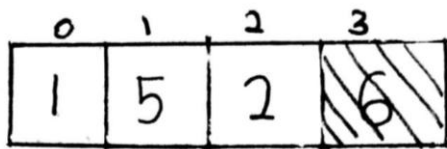
10	7	15	9
----	---	----	---

0	1	2	3
6	5	2	1
pv	i	j	

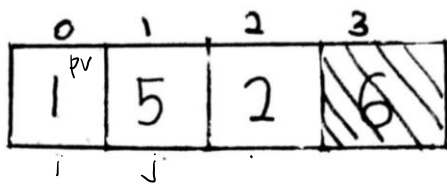
- Start with elements sets less than 7 , pointer **i** is at [0] and pointer **j** is at [1] , our pivot here is 6



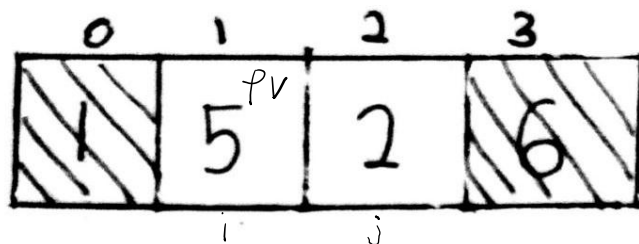
- $5 < 6$ so, i make an increment to $[1]$, $2 < 6$ so i make an increment to $[2]$, $1 < 6$ so i make an increment $[3]$, j will keep move till elements $[3]$



- Next we swap the elements at i with pivot and pivot at $[3]$ is already sorted in its position



- From 3 elements left, we choose the pivot which is 1 as pivot here
- Pointer i at $[0]$ and j at $[1]$
- Now compare, $5 < 1$ so we do not swap, pointer j move to $[2]$
- $2 < 1$ so we do not swap, next we swap elements with pivot which makes elements on $[0]$ locked at the sorted position.



- Choose pivot which is 5 and pointer i at $[0]$ and j at $[1]$
- $2 < 5$ so we do not do anything and this makes 5 and 2 is at its sorted position.

0	1	2	3
1	5	2	6

0	1	2	3
10	7	9	15

0	1	2	3
10	7	15	9
i	j		

- Next we move to sets of elements more or equal to 7 and we choose the pivot as 10
- Pointer **i** is at 10 and pointer **j** is at 7
- $7 < 10$ so increase **i** to [1], **j** moves to 15, $15 > 10$ so we do not swap anything, **j** moves to 9, $9 < 10$ so increase **i** to [2] and swap

0	1	2	3
10	7	15	9
i			
	j		

- We swap the elements on **i** with pivot so this will makes 10 position already in its sorted position

0	1	2	3
9	7	10	15

- 15 is already in sorted as no other elements to compare with on the side of larger than pivot 10.

0	1	2	3
9	7	10	15

- Next set we choose 9 as pivot , pointer **i** at [0] and pointer **j** at [1]

0	1	2	3
9 ^{p_v}	7	10	15

$i \rightarrow j$

- $7 < 9$ so move **i** to [1] , then we swap elements on **i** with pivot

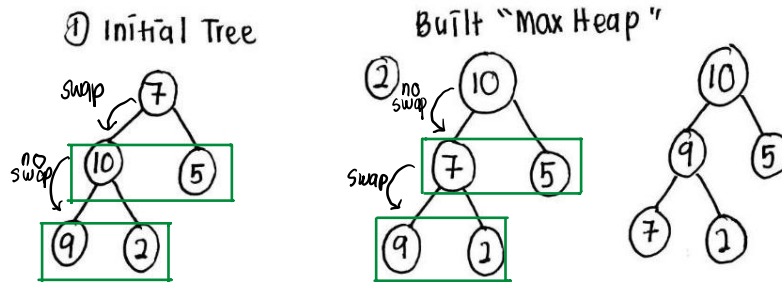
0	1	2	3
7	9	10	15

- This automatically makes 7 and 9 already in sorted position
- Lastly , stitches all the elements back together in order from left to right based on numbers that we have lock and it make final array

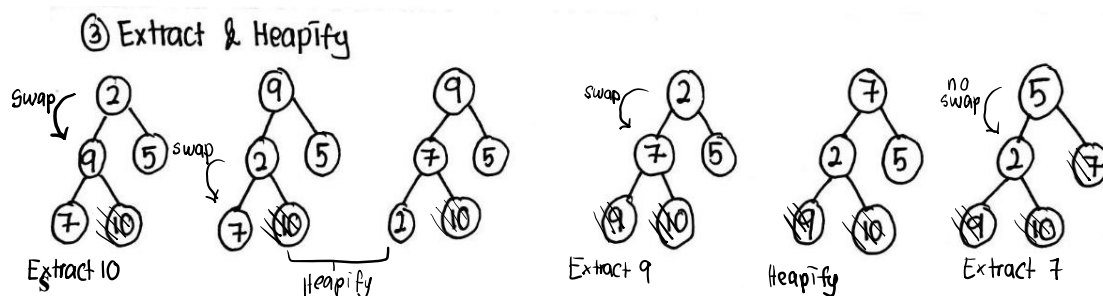
0	1	2	3	4	5	6	7	8
1	2	5	6	7	7	9	10	15

Heap Sort

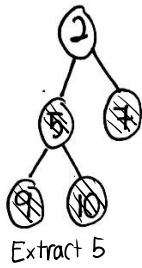
[7 , 10 , 5, 9 ,2]



- Firstly , start by completing binary tree, filling it in from top to bottom, left to right.
- Secondly built "Max Heap" by comparing and swapping to make all the parents be greater than the child
- When we look at binary tree , the lowest parents are 10 so 10 are larger than 9 and 2 so no swapping needed
- Next parents up are 7 , so 7 are smaller the children which is 10 so they must swap
- Then we recheck the lowest parents 7 , 7 is smaller than 9 so they must swap



- Next step is we extract 10 by swapping 10 with elements that are on top of the tree which is 2 , this step will lock and sorted 10 on its place
- Then we heapify it by comparing new root 2 with its child 9 and 5 , so $9 > 2$ and we swap
- Continue comparing 2 with 7 , $7 > 2$ so we do the swapping
- Next extract is 9 , we swap 9 with 2 and lock 9 on its place
- Do the heapify by comparing 2 with 7 and 5 and swap 2 with its largest child
- Extract 7 by swapping with 5 and lock 7 on its place
- Since $5 > 2$ so do not do swapping



- Last extract is 5 so we swap 5 with 2 and lock 5 on its place
- 2 is last elements so it automatically lock into the place
- Our final sorted array is [2,5,7,9,10]

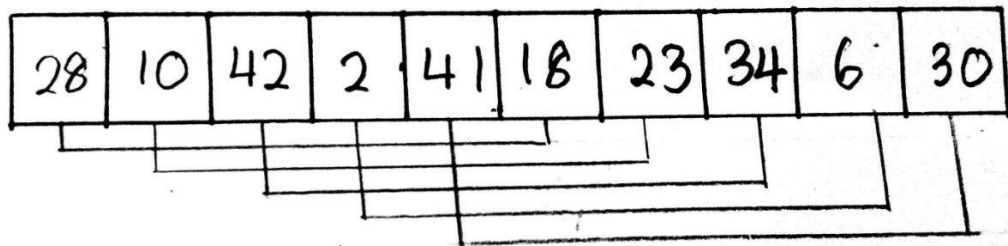
Shell Sort

Initial array : [28,10,42,2,41,18,23,34,6,30]

Array Length (N) : 10

Gap Sequence : Pass 1: $10/2=5$, Pass 2: $5/2=2$ (drop the decimal) , Pass 3: $2/2=1$

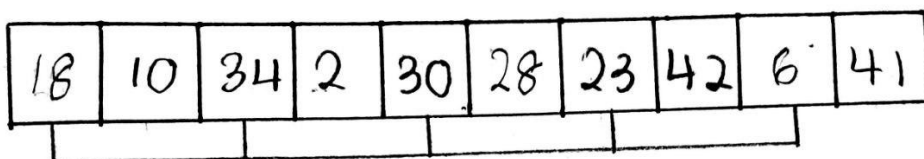
Pass 1



- Compare 28 and 18 , $28 > 18$ so swap
- Compare 10 and 23 . $10 < 23$ so no swap
- Compare 42 and 34 , $42 > 34$ so swap
- Compare 2 and 6 , $2 < 6$ so no swap
- Compare 41 and 30 , $41 > 30$ so swap

Array after pass 1 : [18,10,34,2,30,28,23,42,6,41]

Pass 2



- Split into 2 sublist

18	34	30	23	6
----	----	----	----	---

10	2	28	42	41
----	---	----	----	----

- **Sublist A: [18,34,30,23,6]**
- Compare 34 and 18 , $34 > 18$ so do not swap
- Current array : [18,34,.....]
- Compare 30 and 34 , $30 < 34$ so swap , $34 > 18$ so do not swap
- Current array : [18,30,34,.....]
- Compare 23 and 34 , $23 < 34$ so swap , $23 < 30$ so swap
- Current array : [18,23,30,34,....]
- Compare 6 with all of the number and it is the smallest so 6 will be drag to the front

Sorted sublist A :

6	18	23	30	34
---	----	----	----	----

10	2	28	42	41
----	---	----	----	----

Sublist B: [10,2,28,42,41]

- Compare 2 with 10 , $2 < 10$ so swap
- Current array : [2,10,....]
- Compare 28 with 2 , so $28 > 2$ and $28 > 10$ so do not swap
- Current array : [2,10,28,....]
- Compare 42 with 28 , so $42 > 28$, $42 > 10$ and $42 > 2$ so do not swap 42
- Current array : [2,10,28,42,...]
- Compare 41 with 42 , $41 < 42$ so swap , 41 are large than the number 28,10 and 2 so do not swap

Sorted Sublist B :

2	10	28	41	42
---	----	----	----	----

- Lastly we combine back the sublist A and B into one array

6	2	18	10	23	28	30	41	34	42
---	---	----	----	----	----	----	----	----	----

Pass 3

- Compare 6 and 2 , $6 > 2$ so swap
- Current array : [2,6,...]
- Compare 18 and 6 , $18 > 2$ and $18 > 6$ so do not swap
- Current array : [2,6,18,...]
- Compare 10 with 18 , $10 < 18$ so swap while $10 > 6$ and $10 > 2$ so do not swap
- Current array : [2,6,10,18,...]
- 23 , 28 and 30 are already in their order
- Current array : [2,6,10,18,23,28,30,...]
- Compare 41 and 34 , $41 > 34$ so swap
- Current array : [2,6,10,18,23,28,30,34,41]
- Last element 42 are already in order

Final sorted array

2	6	10	18	23	28	30	34	41	42
---	---	----	----	----	----	----	----	----	----

2. Analyse the Quick Sort, Heap Sort, and Shell Sort algorithm by giving its best case, worst case, space complexity, strength and weakness of this algorithm.

Algorithm	Best Case	Worst Case	Space Complexity	Strength	Weakness
Quick Sort	$O(n \log n)$	$O(n^2)$	$O(\log n)$	Space Efficient : A data being sort in place and only require $O(\log n)$ auxiliary memory for recursive stack	Worst-Case Threat: If it always choose the bad “pivot” , the time complexity will degrade to $O(n^2)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(1)$	Extremely Memory-Safe: The $O(1)$ make it use limited memory as it operate with strict space complexity	Slow practical speed: When algorithm always swap elements in a wide distance within an array ,it defeats hardware caching

					and actually more slower than quick sort
Shell Sort	$O(n \log n)$	$O(n^2)$	$O(1)$	Lighweight & Simple: It uses strict $O(1)$ memory so the code is short and easy to implements	Struggles with massive data: It cannot handle millions of records just as $O(n \log n)$ with a large gap jumps it will inherently unstable sort

3. The recurrence relation $T(n)=T(n-1) +1$ and it's used to find time complexity analysis. Solve the recurrence relation $T(n)=T(n-1) +1$ with the initial condition $T(0) =0$ for $n=4$. You are required to compute the value of $T(4)$. Show your answer in step-by-step process and prove that this result aligns with the closed-form solution $T(n)=n$, which we derived earlier. For $n=4$, substituting into the closed-form solution also gives $T(4)=4$.

For $n = 1$,

$$\begin{aligned}
 T(1) &= T(1-1)+1 \\
 &= T(0)+1
 \end{aligned}$$

$$= 0+1$$

$$= 1$$

For n = 2

$$T(2) = T(2-1)+1$$

$$= T(1)+1$$

$$= 1+1$$

$$= 2$$

For n = 3

$$T(3) = T(3-1)+1$$

$$= T(2)+1$$

$$= 2+1$$

$$= 3$$

For n = 4

$$T(4) = T(4-1)+1$$

$$= T(3)+1$$

$$= 3+1$$

$$= 4$$

: When $T(n) = n$, we substitute $n=4$ so $T(4)=4$.